

FASE 1: GoLite

MANUAL TECNICO

Introducción

El presente documento constituye el Manual Técnico del Interpretador GoLite, desarrollado como proyecto del Laboratorio de Organización de Lenguajes y Compiladores 1. Su propósito es describir con detalle los aspectos técnicos de la implementación, sirviendo como referencia para comprender la arquitectura interna, las decisiones de diseño y el funcionamiento de cada componente del sistema.

GoLite es un lenguaje de programación inspirado en la sintaxis de Go, diseñado con un subconjunto reducido de instrucciones que permite explorar los conceptos fundamentales de la teoría de compiladores: análisis léxico, análisis sintáctico, construcción del AST, análisis semántico e interpretación.

Este manual cubre exclusivamente la Fase 1 del proyecto, que comprende variables y tipos primitivos, operadores, sentencias if/else, bucles for, break/continue y las funciones embebidas `fmt.Println`, `strconv.Atoi`, `strconv.ParseFloat` y `reflect.TypeOf`.

- **Alcance del Documento**

El manual está dirigido a un lector técnico con conocimientos en Java, teoría de compiladores y herramientas de generación de analizadores. Cubre los siguientes temas:

- Entorno de desarrollo y herramientas utilizadas.
- Arquitectura general del sistema y estructura de paquetes.
- Descripción del analizador léxico (JFlex): expresiones regulares y tokens.
- Descripción del analizador sintáctico (CUP): gramática y construcción del AST.
- Descripción de cada clase de nodo del AST
- Funcionamiento del interpretador: evaluación de expresiones y ejecución de sentencias.
- Tabla de símbolos: manejo de scopes y resolución de variables
- Manejo de errores: léxico, sintáctico y semántico.

Entorno de Desarrollo y Herramientas

A continuación, se describen todas las tecnologías y herramientas utilizadas en el desarrollo del proyecto.

- **Lenguaje de Programación: Java 25**

Todo el núcleo del interprete, la interfaz gráfica y la lógica de la tabla de símbolos están implementados en Java 25. Se aprovechan las siguientes características modernas del lenguaje:

- **Pattern marching con instanceof:** permite el dispatch del visitor en el intérprete de forma concisa (`if (node instanceof LiteralNode l) { ... }`).
- **Switch expressions con yield:** utilizados en el método `coerce()` y `evaluateUnary()` del interprete para mayor legibilidad.
- **Text blocks y métodos de String:** usados en el formateo de mensajes de error y salida de consola.

- **JFlex 1.9.1 – Analizador Léxico**

JFlex es un generador de analizadores léxicos para Java. A partir de un archivo `.flex` con expresiones regulares y acciones Java, genera una clase `Lexer.java` que implementa el método `yylex()` para obtener tokens uno a uno.

El archivo `lexer.flex` define: estados del autómata (`YYINITIAL`, `TEXT`, `SIMPLE_COMMENT`, `MULTIPLE_COMMENT`), expresiones regulares para cada categoría de token, y acciones que construyen objetos `Symbol` de CUP y registran entradas en la lista de tokens para el reporte.

- **CUP 11b – Analizador Sintáctico**

CUP (Constructor of Useful Parsers) es un generador de parsers LALR(1) para Java. A partir de un archivo `.cup` con la gramática en BNF y acciones semánticas Java, genera las clases `parser.java` y `sym.java`.

Cada producción de la gramática incluye una acción semántica (`RESULT = ...`) que instancia el nodo AST correspondiente, construyendo el árbol de abajo hacia arriba durante el análisis sintáctico.

- **Apache Maven**

El proyecto utiliza Maven como sistema de construcción. El pom.xml configura los plugins de JFlex y CUP para que se ejecuten automáticamente durante la fase generate-sources de Maven, garantizando que el Lexer.java y parser.java se regeneren antes de cada compilación.

- **NetBeans 28 – IDE de Desarrollo**

El IDE NetBeans se utilizó para el desarrollo, ya que provee integración nativa con Maven, soporte para Java 25, y facilita la configuración de los plugins de JFlex y CUP como parte del ciclo de vida del proyecto.

Herramienta	Versión	Uso en el Proyecto
Java	25	Lenguaje principal del interprete y GUI
JFlex	1.9.1	Generación de analizador léxico
CUP	11b	Generación del analizador sintáctico LALR(1)
Maven	3.x	Sistema de construcción y gestión de dependencias
NetBeans	28	Entorno de desarrollo integrado
Java Swing	(JDK 25)	Biblioteca para la interfaz gráfica de usuario

Arquitectura del Sistema

El intérprete sigue el flujo clásico de un compilador/interprete de una pasada, compuesto por cinco etapas encadenadas: análisis léxico, análisis sintáctico, construcción del AST, análisis semántico e interpretación. La GUI coordina todas estas etapas al presionar el botón Ejecutar.

- **Estructura de Paquetes**

El paquete raíz del proyecto será: **com.mycompany.proyecto_compi1_vj26**.

Para temas de facilidad el paquete **com.mycompany.proyecto_compi1_vj26** se le dará el alias de 'root'.

Paquete	Descripción
root	Clases generadas por JFlex (Lexer) y CUP (parser, sym). Clase Main de la Aplicación.
root.ast	Interfaz ASTNode, clase base BaseNode y el nodo inicial ProgramNode.
root.ast.expressions	Los nodos que representan a las expresiones del lenguaje.
root.ast.statements	Los nodos que representa a las declaraciones del lenguaje.
root.exceptions	Clases que heredan de RuntimeException para el control Break/Continue/Return.
root.frontend	Clases para la Interfaz Gráfica de la Aplicación.
root.models	Clases para los reportes. Enums para Datos Constantes.
root.symbols	Clases Symbol y SymbolTable con gestión de scopes anidados.
root.visitor	Interfaz Visitor.
root.visitor.interpreter	Clase InterpreterVisitor.
root.visitor.interpreter.value	Clases para modelar los diferentes Tipos de Datos.

- **Flujo de Ejecución**

Cuando el usuario presiona el botón Ejecutar en la GUI, se ejecuta la siguiente cadena.

Código fuente (.glt) -> Lexer (JFlex) -> Parser (CUP) -> AST -> Interpreter -> Consola

- El Lexer lee el código carácter a carácter y emite tokens, registrando cada uno en la lista de Token.
- El Parser consume los tokens del Lexer y, usando las acciones semánticas del .cup, construye el AST de abajo hacia arriba.
- Si hay errores léxicos o sintácticos, se detiene la interpretación y se reportan en la tabla de errores.
- El Interpreter recorre el AST de arriba hacia abajo usando el patrón Visitor, ejecutando cada nodo y gestionando la tabla de símbolos.
- Los errores semánticos (tipo incompatible, variable no declarada, etc.) se recolectan y se muestran en el reporte de errores.

Analizador Léxico (JFlex)

El archivo `lexer.flex` define el analizador léxico del lenguaje GoLite. Implementa la lógica de Automatic Semicolon Insertion (ASI), el reporte de tokens y la detección de errores léxicos.

- **Expresiones Regulares**

A continuación, se describen todas las expresiones regulares definidas en el archivo `.flex` y su propósito:

Nombre	Expresión Regular	Descripción
LineTerminator	<code>\r \n \r\n</code>	Salto de línea en cualquier plataforma (Unix, Windows, Mac)
WhiteSpace	<code>[\t\f]</code>	Espacios, tabulaciones y form feed. Se ignoran completamente
InputCharacter	<code>[^\r\n]</code>	Cualquier carácter que no sea salto de línea. Usando en comentarios.
WholeNumber	<code>0 [1-9][0-9]*</code>	Literal entero: el cero exacto, o un dígito no-cero seguido de cualquier cantidad de dígitos. Evita números con cero iniciales como 007.
DecimalNumber	<code>{WholeNumber}{Dot}[0-9]+</code>	Literal decimal: parte entera valida, punto decimal obligatorio, seguido de uno o más dígitos. Ejemplo: 3.14, 0.5
Letter	<code>[a-zA-Z]</code>	Cualquier letra del alfabeto inglés, mayúscula o minúscula.
ID	<code>_?{Letter}({Letter} _{WholeNumber})*</code>	Identificador: inicia con letra o guion bajo, continua con letras, dígitos o guion bajo. Case sensitive.
RuneLiteral	<code>\'([^\r\n\'\\] \\[nrt\"\\] \\u[0-9A-Fa-f]{4})\'</code>	Literal de tipo rune: un carácter simple entre comillas simples, o una secuencia de escape (<code>\n</code> , <code>\t</code> , <code>\r</code> , <code>\\</code> , <code>\'</code>). También acepta escapes Unicode <code>\uXXXX</code> .

- **Estados del Autómata**

JFlex implementa un autómata de estados finitos. El leer de GoLite define cuatro estados:

Estado	Descripción
YYINITIAL	Estado inicial y principal. Reconoce todos los tokens del lenguaje
TEXT	Activo dentro de un literal string (entre comillas dobles). Maneja secuencias de escape y detecta strings sin cerrar.
SIMPLE_COMENT	Activo después de //. Ignora todo hasta el salto de línea. Al salir puede insertar un punto y coma automático (ASI).
MULTIPLE_COMENT	Activo entre /* y */. Ignora todo el contenido. Detecta el caso de comentario sin cerrar al llegar a EOF.

- **Automatic Semicolon Insert (ASI)**

GoLite hereda de Go el mecanismo de inserción automática de punto y coma. El lexer no requiere que el programador escriba ; al final de cada línea. En su lugar, el Lexer inserta automáticamente un token PUNTO_COMA cuando:

- Se encuentra un salto de línea (LineTerminator) o fin de archivo (EOF), Y
- El ultimo token emitido fue uno de: ID, literal numérico, literal string, literal rune, true, false, return, break, continue, ++, --,),] o }.

Esto se implementa mediante el método updateASI(int type) que actualiza el flag insertSemicolon después de cada token emitido, y las reglas de LineTerminator que verifican ese flag antes de decidir si insertar el punto y coma.

Analizador Sintáctico (CUP)

El archivo parser.cup define la gramática LALR(1) del lenguaje GoLite y las acciones semánticas que construyen el AST. CUP genera las clases parser.java (el parser) y sym.java (los identificadores numéricos de cada terminal y no-terminal).

- Declaración de Precedencias**

Las precedencias se declaran de menor a mayor. CUP usa esta información para resolver conflictos shift/reduce en la gramática de expresiones:

Home	Operadores	Asociatividad y nota
1 (menor)		Izquierda. Cortocircuito: si el operando izquierdo es true, el derecho no se evalúa.
2	&&	Izquierda. Cortocircuito: si el operando izquierdo es false, el derecho no se evalúa.
3	!	Derecha, Negación y desigualdad.
4	== !=	Izquierda, Igualdad y desigualdad.
5	< <= > >=	Izquierda, Comparaciones relacionales.
6	+ -	Izquierda. Suma y resta.
7	/ *	Izquierda. División y multiplicación
8	%	Izquierda. Modulo.
9 (mayor)	Uminus	Derecha. Negación aritmética unario. Token virtual declarado solo para precedencia.

- Construcción del AST en Acciones Semánticas**

Cada producción del parser incluye una acción semántica que instancia el nodo AST correspondiente y lo asigna a RESULT. CUP propaga estos valores de abajo hacia arriba en el árbol de derivación. El símbolo inicial inicio retorna un ProgramNode que es la raíz del AST completo.

Ejemplo de acción semántica para una declaración de variable:

```
variable ::= VAR:v ID:name tipo_dato:t definición:d
```

```
{: RESULT = new VarDecl(name, t, d, vleft, vright); :}
```

Las variables `vleft` y `vright` son generadas automáticamente por CUP y contienen la línea y columna del token `VAR`, usadas para el reporte de errores.

Árbol de Sintaxis Abstracta (AST)

El AST está compuesto por 20 clases de nodo organizadas en el paquete `com.mycompany.proyecto_compi1_vj26.ast`. Todas implementan la interfaz `ASTNode` y extienden la clase abstracta `BaseNode` que provee los campos `line` y `column`.

- **Jerarquía de Clases**

Clase	Descripción y campos principales
ProgramNode	Raíz del AST. Contiene la referencia a la función main (FuncDeclNode).
FuncDecl	Declaración de función. Campos: name (String), body (Block). Fase 1: solo main.
Block	Bloque { }. Contiene una List<AST> de sentencias. Abre y cierra un scope.
VarDecl	Declaración var x tipo = expr. Campos: name, type, initValue (puede ser null).
ImplicitVarDecl	Declaración x := expr. El tipo se infiere del valor en tiempo de ejecución.
Assign	Asignación x = expr. Verifica que la variable exista y que el tipo sea compatible.
ImplicitAssign	Asignación compuesta x += expr / x -= expr. Campo operator: "+=", "-=".
IncDec	Incremento/decremento i++ / i--. Solo valido para int y float64.
If	if/else if/else. Campos: condition, thenBlock, lista de elseifConditions/Blocks, elseBlock.
ForCond	for condicion { }. Campos: condition (debe ser bool), body (BlockNode).
ForClassic	for init; cond; inc { }. Campos: init, condition, increment, body.
Break	Sentencia break. Solo valido dentro de un bucle. Lanza BreakException.
Continue	Sentencia continue. Solo valido dentro de un bucle. Lanza ContinueException.
Binary	Expresión binaria. Campos: left, operator (String), right.
Unary	Expresión unaria. Campos: operator ("-" o "!"), operand.
Literal	Literal primitivo. Campos: value (ValueWrapper), type ("int", "float64", "string", "bool", "rune", "nil").

Identifier	Acceso a variable. Campo: name. El intérprete lo resuelve buscando en los scopes.
FuncCall	Llamada a función embebida. Campos: functionName, List<ASTNode> args.

Tabla de Símbolos

La tabla de símbolos esta implementada en el paquete `com.mycompany.proyecto_compi1_vj26.symbols` con dos clases: `Symbol` y `SymbolTable`.

- **Clase Symbol**

Representa una variable declarada. Sus campos son:

- `name (String)`: nombre del identificador.
- `type (VarType)`: tipo estático declarado (`int`, `float64`, `string`, `bool`, `rune`).
- `value (ValueWrapper)`: valor actual en tiempo de ejecución (`IntVale`, `DoubleValue`, `StringValue`, `BoolValue`, `CharValue`, `NullValue`).
- `scope (int)`: nivel de anidamiento en que fue declarada (0 = funcion main, 1+ = bloques anidados).
- `line / column (int)`: posición de la declaración para el reporte de errores.

- **Clase SymbolTable**

Implementa la pila de scopes usando una `List<LinkedHashMap<String, Symbol>>`. Los `LinkedHashMap` preservan el orden de inserción, lo que facilita la generación del reporte de tabla de símbolos.

Método	Comportamiento
<code>pushScope()</code>	Agrega un nuevo <code>LinkedHashMap</code> a la pila. Se llama al entrar a cada bloque <code>{ }</code> .
<code>popScope()</code>	Elimina el mapa del tope. Se llama al salir de cada bloque. Las variables locales dejan de ser visibles.
<code>declare()</code>	Declara una variable en el scope actual. Si ya existe en ese scope (mismo nivel), reporta error semántico.
<code>lookup()</code>	Busca de adentro hacia afuera. Si hay shadowing, retorna la variable del scope más interno.
<code>assign()</code>	Actualiza el valor en el scope donde fue declarada la variable, no en el scope actual.
<code>reset()</code>	Limpia toda la pila y los contadores. Se llama al inicio de cada nueva ejecución desde la GUI.

Visitor (Interpreter)

La clase VisitorInterpreter en el paquete `com.mycompany.proyecto_compi1_vj26.visitor.interpreter` es el núcleo de ejecución. Implementa el patrón Visitor en Java. El método principal es:

- `visit(Context)`: visita los nodos de sentencias y expresiones retornando un `ValueWrapper` (`IntValue`, `DoubleValue`, `StringValue`, `BoolValue`, `CharValue`, `NullValue`, `VoidValue`).

• Reglas de Tipos en Operaciones Binarias

Operando izquierdo	Operando derecho	Resultado
int	int	int (division entera para /)
int	float64	float64 (promoción implícita)
float64	int	float64 (promoción implícita)
float64	float64	float64
string	string	string (solo con operador +, concatenación)
rune	int	int (el rune se trata como su valor Unicode)

• Transferencia de Control con Excepciones

El intérprete usa tres excepciones de control que extienden `RuntimeException`. Se usan como mecanismo de transferencia de control, NO como errores:

- `BreakException`: lanzada por `executeBreak()`. Capturada en `executeForCond()` y `executeForClassic()` para salir del bucle.
- `ContinueException`: lanzada por `executeContinue()`. Capturada en el cuerpo del bucle para saltar al siguiente ciclo (y ejecutar el incremento en el for clásico).
- `ReturnException`: lanzada cuando se encuentra un nodo `return`. Capturada en `executeFunc()` para detener la ejecución de la función. Lleva un campo `value` con el valor de retorno (null en Fase 1).

Las tres excepciones se construyen con `super(null, null, true, false)`, deshabilitando el llenado del `stack trace` para evitar overhead innecesario.

Manejo de Errores

El sistema de errores esta centralizado en el paquete `com.my.company.proyecto_compi1_vj26.models` con dos clases de datos: `Token` y `ErrorReport`.

- **Clases de Datos**

- **Token:** almacena lexeme, type (), line, column de cada token reconocido.
- **ErrorReport:** almacena description, line, column y type (enum: LEXICO, SINTACTICO, SEMANTICO).

- **Comportamiento por Tipo de Error**

Tipo	Origen	Comportamiento
Léxico	Caracter no reconocido, string sin cerrar, comentario sin cerrar.	Se registra en lexicalErrors del Lexer.
Sintáctico	Token inesperado, estructura gramatical invalida.	Se registra en syntaxErrors del parser via syntax_error(). Se intenta recuperación con la regla 'error'.
Semántico	Variable no declarada, re declaración, tipo incompatible, división por cero, break/continue fuera de bucle.	Se registra en semanticErrors del Interpreter o de SymbolTable. La ejecución continua cuando es posible.

Interfaz Gráfica (GUI)

La GUI esta implementada en el paquete `com.mycompany.proyecto_compi1_vj26.frontend` con cuatro clases Swing.

- **Descripción de Clases**

Clase	Responsabilidad
IDEGoLite	Ventana principal. Coordina la barra de menus, el JTabbedPane, la consola y la ejecución del interprete. Maneja abrir/guardar archivos .glt.
EditorTab	Panel de una pestaña del editor. Contiene el JTextArea principal con otro JTextArea como row header del JScrollPane. Mantiene el archivo asociado y el flag de modificado.
TabHeader	Componente de encabezado personalizado de pestaña con JLabel del título y botón X para cerrar. El botón cambia a rojo en hover.
ReportFrame	JFrame reutilizable para reportes. Recibe el modo (TOKENS o ERRORS) y una lista de datos. Colorea filas de errores según su tipo.

- **Ciclo de Ejecución en la GUI**

El método ejecutar() de IDEGoLite sigue estos pasos al presionar el botón de Ejecutar o F5:

1. Obtiene el texto del EditorTab activo.
2. Instancia el Lexer con un StringReader del texto.
3. Instancia el parser pasando el Lexer. Llama a `p.parse()` para obtener el ProgramNode.
4. Si hay errores léxicos o sintácticos: muestra el resumen en consola, guarda los errores para el reporte.
5. Si el AST es válido: instancia el VisitorInterpreter y llama a `interpreter.visit(ast)`.
6. Muestra la salida del interprete en la consola y registra los errores semánticos para el reporte.